

TP - Session 13 : Testing React

Module M204 - Developpement Front-End

Objectif : Ecrire des tests unitaires pour les composants TechShop.

Duree : 1h30 | **Projet :** tp-styling-techshop (session precedente)

Mise en Place

1. Ouvrir le projet TechShop
2. Installer les dependances de test :

```
npm install -D vitest @testing-library/react @testing-library/jest-dom jsdom
```

3. Ajouter dans vite.config.js :

```
test: {
  globals: true,
  environment: 'jsdom',
  setupFiles: './src/setupTests.js',
}
```

4. Creer src/setupTests.js :

```
import '@testing-library/jest-dom'
```

5. Ajouter dans package.json scripts :

```
"test": "vitest",
"test:run": "vitest run"
```

1 Tester une Fonction Utilitaire (15 pts)

Creer src/utils/helpers.js avec les fonctions :

```
export function formatPrice(price) {
  return `${price} DH`
}

export function getStockStatus(stock) {
  if (stock > 10) return 'En stock'
  if (stock > 0) return 'Stock faible'
  return 'Rupture'
}
```

Creer src/utils/helpers.test.js et ecrire les tests pour :

- formatPrice(100) retourne '100 DH'
- getStockStatus(15) retourne 'En stock'
- getStockStatus(5) retourne 'Stock faible'
- getStockStatus(0) retourne 'Rupture'

Aide

```
import { describe, it, expect } from 'vitest'
```

2 Tester un Composant Simple (20 pts)

Créer `src/components/Button.jsx` :

```
function Button({ children, onClick, variant = 'primary' }) {
  return (
    <button className={`btn btn-${variant}`} onClick={onClick}>
      {children}
    </button>
  )
}
export default Button
```

Créer `src/components/Button.test.jsx` et tester :

- Le bouton affiche le texte passe en children
- Le bouton a le role "button"
- `onClick` est appelé quand on clique (utiliser `vi.fn()`)

Aide

```
import { render, screen, fireEvent } from '@testing-library/react'
import { vi } from 'vitest'
```

3 Tester un Composant avec State (25 pts)

Créer `src/components/Counter.jsx` :

```
import { useState } from 'react'

function Counter({ initialValue = 0 }) {
  const [count, setCount] = useState(initialValue)

  return (
    <div>
      <span data-testid="count">{count}</span>
      <button onClick={() => setCount(count + 1)}>+</button>
      <button onClick={() => setCount(count - 1)}>-</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  )
}
export default Counter
```

Créer `src/components/Counter.test.jsx` et tester :

- Affiche 0 par défaut
- Affiche `initialValue` si fourni

- Incrémente quand + clique
- Décrémente quand - clique
- Reset à 0 quand Reset clique

Aide

`screen.getByTestId('count')` pour trouver le span
`screen.getText('+')` pour trouver le bouton +

4 Tester ProductCard (25 pts)

Créer `src/components/ProductCard.test.jsx` pour tester le composant existant :

```
const mockProduct = {  
  id: 1,  
  name: 'Test Product',  
  price: 999,  
  category: 'Tech',  
  image: 'test.jpg'  
}
```

Tester :

- Affiche le nom du produit
- Affiche le prix avec "DH"
- Affiche la catégorie
- `onEdit` est appelé avec l'id quand Modifier clique
- `onDelete` est appelé avec l'id quand Supprimer clique

5 Bonus : Tester un Formulaire (15 pts)

Tester `ProductForm.jsx` :

- Le formulaire contient les champs (par placeholder)
- `onSubmit` est appelé avec les données saisies

Aide pour remplir un input

```
fireEvent.change(screen.getByPlaceholderText('Nom'), {  
  target: { value: 'iPhone' }  
})
```

Lancer les tests : `npm test`

Voir la couverture : `npm test -- --coverage`